PERSONAL LEARNING PLAN · OCTOBER 2026

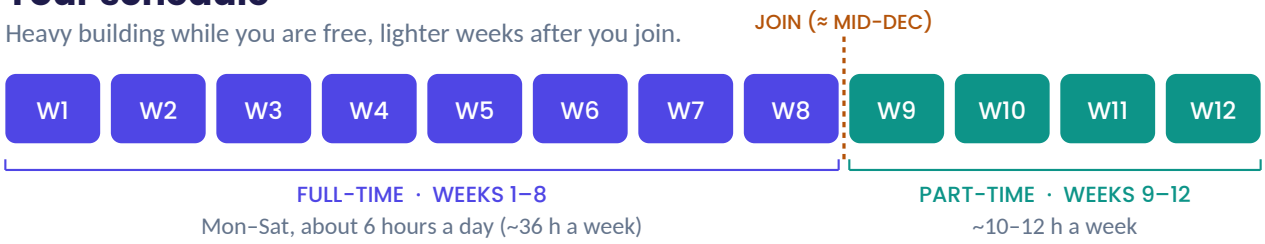
AI Agents 12-Week Plan

From multi-agent builder to production agent engineer

Built around your resume and checked against the agent ecosystem as of October 2026: LangChain/LangGraph 1.x, MCP spec 2026-07-28, A2A v1.0 and the OWASP Top 10 for Agentic Applications.

Your schedule

Heavy building while you are free, lighter weeks after you join.



Suggested dates: start Mon 19 Oct 2026 · Week 8 ends Sun 13 Dec · Week 12 ends Sun 10 Jan 2027. Shift if you start later.

By the end of Week 12 you will have

- ✓ A deployed multi-agent research assistant with evals, tracing, security tests and a cost report
- ✓ A remote, OAuth-protected MCP server on the 2026-07-28 spec, and an agent connected over A2A
- ✓ Honest, measured numbers for your resume and a public write-up of what you learned
- ✓ Clear answers to the questions agent-engineering interviews and design reviews ask

Inside every week

Goal → Day-by-day steps → Deliverable → Self-check → Resources

Weeks 1–8 are planned day by day; Weeks 9–12 are planned in five sessions each. Every resource link is clickable.

READ THIS FIRST

How to use this plan

You have about 8 free weeks before you join, then about 4 weeks of evenings and weekends. So the plan puts the heavy building into Weeks 1–8 and keeps Weeks 9–12 lighter. Every week has the same layout: **goal → day-by-day steps → deliverable → self-check → resources**.

Daily rhythm in Weeks 1–8 (Mon–Sat, about 6 hours)

LEARN · 2 h	Read or watch the day's resources. Take notes in your own words, not copy-paste.
BUILD · 3.5 h	Code the day's tasks. Commit small and often. When stuck for 45 minutes, simplify the task.
LOG · 30 min	Write 5–10 lines in `LEARNINGS.md`: what worked, what broke, what surprised you. Push.

Weekly rhythm

- **Days 1–5:** follow the day plan.
- **Day 6:** finish the deliverable, write the README, and answer the self-check in writing. Day 6 is also your buffer day for anything that slipped.
- **Day 7:** rest. Spend 20 minutes on a review: what slipped, what carries over, and a quick look at the LangChain changelog, the MCP blog and the Anthropic Engineering blog.
- **Weeks 9–12:** each week is split into 5 sessions (about four 1-hour weekday evenings and two 3–4 hour weekend blocks).

Ground rules

- **Build more than you watch.** At least 60% of your hours go into code.
- **A week is done** only when the repo is pushed with a README and the self-check is answered.
- **Measure from Day 1.** Log tokens, cost and latency for every LLM call.
- **Cut scope, not the deliverable.** Ship a smaller version and move on.
- **Festival weeks or sick days:** use Day 6 as a buffer and push the rest forward. Never skip Week 8.

Before Day 1: setup checklist

- ☐ Python 3.12, uv, Docker, Git and your editor installed.
- ☐ One GitHub repo, `agent-engineering-12w`, with a folder per week and a `LEARNINGS.md` at the root.
- ☐ An API key for one main provider (Anthropic, OpenAI or Google), plus OpenRouter for cheap experiments. Optional: Ollama for free local models.
- ☐ A hard monthly spending limit set in every provider console.
- ☐ `.env` listed in `.gitignore`, and pre-commit with a secret scanner such as gitleaks.
- ☐ Free accounts: LangChain Academy, and a Google AI Studio key for Week 6.

Keeping API costs low

- Develop with small, cheap models. Use strong models only for final runs and evals.
- Cache LLM responses during development (keyed by a hash of the prompt) so re-runs are free.
- Keep a running cost total in your JSONL log and check it every Sunday.

BASED ON YOUR RESUME

Your starting point

You are at **Level 3 of 5** and already have one foot in Level 4. This plan takes you to a solid Level 4: you will be able to prove, with traces and eval numbers, that your agents work, what they cost, and how they stay safe.

LEVEL	WHAT IT MEANS	YOU TODAY
1 · LLM user	APIs, prompting, chatbots	Done
2 · Tool-using agent	ReAct, tools, RAG	Done: Travel Planner
3 · Multi-agent builder	Several agents with a router or supervisor	Done: Octra (5 agents)
4 · Production agent engineer	Evals, tracing, security, crash-resume, cost control	Halfway: deployment, fallbacks and escalation exist; evals, tracing and security don't yet
5 · Agent architect	Large system design, protocols, agents that run for hours	Not yet (normal for a fresher)

The highlighted row is your target for this plan.

STRENGTHS TO BUILD ON

- Octra: a LangGraph multi-agent RAG system with 5 specialist agents, routing, multilingual replies, streaming, escalation and a fallback chain.
- A ReAct travel planner with 5 tools, and a Claude + MCP expense tracker.
- Strong deployment skills: Docker, Kubernetes (EKS), GitHub Actions, FastAPI, Redis, MLflow, DVC and Prometheus/Grafana. Few agent learners have these.

GAPS THIS PLAN CLOSES

- No measured quality numbers (evals) for your agents yet.
- No agent-level tracing: which tool, why, and how many tokens.
- Multi-agent work stops at routing: no parallel subagents, handoffs, human approval or crash-resume.
- No remote MCP server on the 2026-07-28 spec with OAuth shown yet, and no A2A.
- No visible security work (prompt injection, tool permissions) or cost control.

THE ONE SHIFT THAT MATTERS IN 2026

A Level 3 engineer says "my agent works". A Level 4 engineer shows it: "it succeeds 95% of the time on our test set, one run costs this much, and here is how it stays safe when someone attacks it." Every week of this plan pushes you toward the second sentence.

OVERVIEW

12 weeks at a glance

Click a week number to jump to its page. FT = full-time (before joining), PT = part-time (after joining).

WK	DATES	MODE	THEME	KEY DELIVERABLE
1	19–25 Oct	FT	**Agents from scratch**	Agent loop + the 5 workflow patterns, with cost and latency numbers
2	26 Oct–1 Nov	FT	**Tools, context, memory**	Long-task agent with compaction, memory assistant, agentic RAG comparison
3	2–8 Nov	FT	**MCP deep dive**	Remote, OAuth-protected MCP server + your own MCP client
4	9–15 Nov	FT	**LangGraph 1.x in production**	Migrated projects + Octra v2 with persistence and human approval
5	16–22 Nov	FT	**Multi-agent patterns**	Pattern library + three-way architecture bake-off
6	23–29 Nov	FT	**ADK, A2A, capstone design**	ADK agent over A2A + capstone design doc + skeleton
7	30 Nov–6 Dec	FT	**Capstone build**	Capstone v1 running locally with full tracing
8	7–13 Dec	FT	**Evals & observability**	Eval suite + CI gate + before/after numbers
9	14–20 Dec	PT	**Security & guardrails**	Threat model + attack test suite in CI
10	21–27 Dec	PT	**Cost, reliability, deploy**	Live deployment, dashboards and a cost report
11	28 Dec–3 Jan	PT	**Portfolio & visibility**	README, demo video, blog post, updated resume
12	4–10 Jan	PT	**Review & next steps**	3 system designs, a work proposal, next-quarter plan

The orange line marks your joining date (about mid-December).

HOW THE WEEKS CONNECT

- **Weeks 1–2** give you the fundamentals that every framework hides.
- **Weeks 3–6** add the 2026 standards (MCP, LangGraph 1.x, A2A) and the multi-agent patterns.
- **Weeks 7–8** turn everything into one capstone and prove it works with evals.
- **Weeks 9–12** make it safe, cheap and visible, then point you at what comes next.

WEEK 01

Agents From Scratch (No Framework)

FULL-TIME · ~36 h
Suggested: 19–25 Oct

Goal. Understand exactly what happens inside an agent by building the loop, tool calling, structured output and the five workflow patterns in plain Python. No LangChain this week.

Why it matters: Frameworks hide the loop. When an agent misbehaves in production, you need to know what the model saw, what it asked for, and what you sent back.

DAY 1 **Foundations & first tool call**	• Read *Building Effective Agents* twice: once quickly, once with notes. Write half a page: workflow vs agent, and when you need neither. • Set up the repo (uv, .env, pre-commit). Write a small `llm.py` wrapper that logs every call (model, input/output tokens, latency, cost) to a JSONL file. You will reuse it for all 12 weeks. • Make your first raw tool call with your provider's SDK (one fake `get_weather` tool). Print the full request and response JSON and label every field: roles, tool-call IDs, stop reason.
DAY 2 **The agent loop**	• Write `run_agent(task, tools, max_steps)`: call the model → if it asks for tools, run them (in parallel when there are several) → send results back → repeat until it stops or hits max_steps. • Handle the failure paths: unknown tool, bad arguments, tool exceptions (send the error text back so the model can recover), timeouts, and a hard stop. • Add three real tools: a calculator, Open-Meteo weather (free, no API key) and a local file search.
DAY 3 **Structured output & streaming**	• Define Pydantic models and get validated structured output from your provider. On a validation error, retry once and include the error message. • Build an extractor: messy text (an email or invoice) → typed object. Write 10 test inputs and count how many come out right. • Stream tokens to the terminal and see how tool-call chunks arrive in a stream.
DAY 4 **Workflow patterns 1–3**	• Prompt chaining with a gate: outline → check → draft. The gate is plain Python, not an LLM. • Routing: classify requests into 4 categories, then call a specialised prompt. Hand-label 30 requests and measure routing accuracy. This is your first eval. • Parallelisation: sectioning (split a long document and process the parts concurrently with asyncio) and voting (run the same check 3 times and take the majority).
DAY 5 **Patterns 4–5 + autonomous agent**	• Orchestrator-workers: the orchestrator decides subtasks at runtime, dispatches workers and merges their results. • Evaluator-optimizer: a generator writes, an evaluator critiques against a rubric; loop until it passes or 3 rounds are used. • Solve the same task with a fixed workflow and with your Day-2 agent. Compare quality, cost and latency from your JSONL log.
DAY 6 **Mini-project & wrap-up**	• Rebuild your Travel Planner's core without LangChain: 5 tools (mock flights and hotels where APIs are paid) plus re-planning when a tool fails. • README: one section per pattern with a diagram, when to use it, when not to, and the cost and latency you observed. • Answer the self-check in writing. Push.

DELIVERABLE

Repo `week01-agents-from-scratch`: logging LLM wrapper, agent loop, the 5 patterns, Travel Planner rewrite, and a README with measured cost and latency.

SELF-CHECK: YOU ARE DONE WHEN YOU CAN…

- ☐ Write a tool-calling agent loop from memory in under 30 minutes.
- ☐ Explain workflow vs agent with one example of each from your own code.
- ☐ State your router's accuracy and the mistakes it made.

Resources

READ	**Building Effective Agents** · Anthropic	https://www.anthropic.com/engineering/building-effective-agents
CODE	**Agent pattern notebooks (reference code for the 5 patterns)** · Claude Cookbooks	https://github.com/anthropics/claude-cookbooks/tree/main/patterns/agents
DOCS	**Tool use overview** · Claude docs	https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview
DOCS	**Function calling guide** · OpenAI docs	https://platform.openai.com/docs/guides/function-calling
PAPER	**ReAct: Synergizing Reasoning and Acting in Language Models** · Yao et al., 2022	https://arxiv.org/abs/2210.03629
DOCS	**Pydantic: structured outputs and validation** · Pydantic	https://docs.pydantic.dev
COURSE	**Agentic AI, Modules 1–3 (optional; certificate needs Pro)** · Andrew Ng, DeepLearning.AI	https://www.deeplearning.ai/courses/agentic-ai/
TOOL	**Ollama: free local models for loop experiments (optional)** · Ollama	https://ollama.com

MY NOTES FOR THIS WEEK

WEEK 02

Tools, Context Engineering & Memory

FULL-TIME · ~36 h
Suggested: 26 Oct–1 Nov

Goal. Design tools that agents use correctly, manage the context window on purpose, and give agents short- and long-term memory.

Why it matters: Many agent bugs are really context bugs: the model had the wrong information, too much of it, or not enough. Good tools and clean context often fix more than prompt tweaks.

DAY 1 **Tool design**	• Read *Writing effective tools for agents* and *Advanced tool use* (tool search and programmatic tool calling). • Take 3 deliberately bad tools (vague names, overlapping jobs, huge outputs, cryptic errors) and redesign them: clear names and descriptions, input examples, a concise/detailed response option, pagination, and errors that tell the model what to do next. • Measure tool-selection accuracy on 25 test prompts before and after. Record both numbers.
DAY 2 **Context engineering: theory**	• Read Anthropic's *Effective context engineering for AI agents*, LangChain's *Context Engineering for Agents*, Drew Breunig's *How to Fix Your Context* and Chroma's *Context Rot*. • Write a 1-page cheat sheet in your own words: the four strategies (write, select, compress, isolate), the common failure modes (poisoning, distraction, confusion, clash) and their fixes. • Add one example from your past projects where each strategy would have helped.
DAY 3 **Context engineering: practice**	• Build a long-task agent that analyses 30–50 files (CSVs or notes) and writes a report. • Add token counting per step, auto-summarisation above a threshold, a `NOTES.md` the agent writes and re-reads, clearing of old tool results, and just-in-time loading (tools that fetch details instead of preloading everything). • Run with and without these features. Compare success, tokens and cost.
DAY 4 **Memory**	• Learn the types: short-term (thread state), long-term semantic (facts), episodic (past events) and procedural (instructions and skills). • Build an assistant with long-term memory: extract facts after each turn → store them in Qdrant with metadata and timestamps → retrieve relevant memories at the start of each turn. • Test the hard cases with 10 scripted conversations: a fact that changes ("I now prefer short answers"), a delete request, and duplicates.
DAY 5 **Agentic RAG**	• Upgrade naive RAG so the agent decides whether to retrieve, rewrites queries, does multi-hop retrieval, grades relevance and cites sources. • Use documents you can judge (for example the LangGraph docs). Write 25 question–answer pairs and compare naive vs agentic RAG on accuracy, citations and cost.
DAY 6 **Skills + wrap-up**	• Read the Agent Skills post. Create 3 `SKILL.md` files and a loader that shows only their names and descriptions until the agent asks for one (progressive disclosure). • README with all before/after numbers. Self-check. Push.

DELIVERABLE

Repo `week02-context-memory`: tool redesign experiment, long-task agent with compaction, memory assistant, agentic RAG comparison and a skills loader, each with numbers.

SELF-CHECK: YOU ARE DONE WHEN YOU CAN…

- ☐ Name the four context strategies and show one example of each in your code.
- ☐ Explain when to summarise history, when to clear old tool results, and when to move work into a subagent.
- ☐ Show your assistant updating a changed fact instead of storing a duplicate.

Resources

READ	**Writing effective tools for agents, with agents** · Anthropic	https://www.anthropic.com/engineering/writing-tools-for-agents
READ	**Advanced tool use: tool search and programmatic tool calling** · Anthropic	https://www.anthropic.com/engineering/advanced-tool-use
READ	**Effective context engineering for AI agents** · Anthropic	https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents
READ	**Context Engineering for Agents** · LangChain	https://www.langchain.com/blog/context-engineering-for-agents
READ	**How to Fix Your Context** · Drew Breunig	https://www.dbreunig.com/2025/06/26/how-to-fix-your-context.html
PAPER	**Context Rot: how input length changes model performance** · Chroma Research	https://research.trychroma.com/context-rot
READ	**Equipping agents for the real world with Agent Skills** · Anthropic	https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills
DOCS	**LangGraph persistence: threads, checkpoints, memory store** · LangChain docs	https://docs.langchain.com/oss/python/langgraph/persistence
PAPER	**MemGPT: Towards LLMs as Operating Systems** · Packer et al., 2023	https://arxiv.org/abs/2310.08560
DOCS	**Mem0: a ready-made memory layer to compare with yours (optional)** · Mem0	https://docs.mem0.ai

MY NOTES FOR THIS WEEK

WEEK 03

MCP Deep Dive (2026-07-28 Spec)

FULL-TIME · ~36 h
Suggested: 2–8 Nov

Goal. Build production-quality MCP servers and a client on the current stateless spec, with OAuth, and learn where MCP fits and where it doesn't.

Why it matters: MCP is now the standard way to connect agents to tools and data. Building servers, not just using them, is what employers ask for.

HEADS-UP
The 2026-07-28 spec removed protocol sessions and the initialize handshake, and the MCP Python SDK v2 (released 28 July 2026) renamed FastMCP to MCPServer. Many 2025 tutorials are now out of date, so trust the official docs first.

DAY 1 **Concepts & the new spec**	• Read the MCP docs: host, client and server; tools vs resources vs prompts; stdio vs Streamable HTTP. • Read the 2026-07-28 release-candidate post and the AAIF migration guide. Make a table of what was removed (sessions, handshake), what is deprecated (Roots, Sampling, Logging) and what is new (stateless core, Extensions, MCP Apps, Tasks, auth changes). • Skim the Python SDK v2 release notes and its migration guide.
DAY 2 **Local MCP server**	• Install `mcp[cli]` and build your Expense Tracker as an `MCPServer`: tools (`add_expense`, `list_expenses`, `category_summary`, `forecast`), one resource (monthly report) and one prompt (monthly review). • Apply Week 2's tool design: clear descriptions, read-only annotations, input validation. • Test every tool in MCP Inspector, then connect the server to at least two MCP clients (for example Claude Desktop and VS Code).
DAY 3 **Remote & stateless**	• Switch to Streamable HTTP. Keep the server stateless: all state lives in Postgres (or Supabase), keyed by user. • Containerise it. Run two replicas behind a simple load balancer to prove that any instance can serve any request. • Add pytest tests for every tool.
DAY 4 **Authorization (OAuth 2.1)**	• Read the MCP authorization spec: protected resource metadata, authorization-server discovery, audience-bound tokens and Client ID Metadata Documents (Dynamic Client Registration is deprecated). • Run Keycloak in Docker as your authorization server. Validate the token on every request and map scopes to tools (read vs write). • Prove that user A can never read user B's expenses.
DAY 5 **Your own client + long jobs**	• Use the SDK's `Client` to connect your Week 1 agent loop to two or more MCP servers: list tools, namespace them, route calls and react to tool-list changes. • Make `forecast_6_months` a long-running job: return a handle and let the client poll for status. Use the Tasks extension if your SDK version supports it (v2.0.0 shipped without it); otherwise build the same pattern with a jobs table.
DAY 6 **Security pass + wrap-up**	• Check your server for common MCP risks: over-broad tools, token passthrough, missing audience checks, and instructions hidden inside tool results. • Optional: a small MCP Apps UI (a monthly spend chart). • README with an auth-flow diagram and connection steps. Self-check. Push.

DELIVERABLE

Repo `week03-mcp`: a remote, stateless, OAuth-protected MCP server with tests and Docker, plus your own MCP client wired into your agent loop.

SELF-CHECK: YOU ARE DONE WHEN YOU CAN…

- ☐ Explain “stateless at the protocol layer” and say where your server’s state lives now.
- ☐ Explain tools vs resources vs prompts and why you chose each one.
- ☐ Walk through your OAuth flow and show how cross-user access is blocked.

Resources

DOCS	**MCP documentation (start here)** · modelcontextprotocol.io	`https://modelcontextprotocol.io`
DOCS	**MCP specification, revision 2026-07-28** · MCP	`https://modelcontextprotocol.io/specification/2026-07-28`
DOCS	**Authorization (2026-07-28)** · MCP	`https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization`
READ	**The 2026-07-28 release candidate, explained** · MCP blog	`https://blog.modelcontextprotocol.io/posts/2026-07-28-release-candidate/`
READ	**MCP 2026-07-28: what’s changing and how to migrate** · Agentic AI Foundation	`https://aaif.io/blog/mcp-2026-07-28-whats-changing-and-how-to-migrate`
CODE	**MCP Python SDK v2.0.0 release notes (links to the migration guide)** · GitHub	`https://github.com/modelcontextprotocol/python-sdk/releases/tag/v2.0.0`
TOOL	**MCP Inspector (test and debug servers)** · GitHub	`https://github.com/modelcontextprotocol/inspector`
DOCS	**Using Keycloak as an MCP authorization server** · Keycloak	`https://www.keycloak.org/nightly/securing-apps/mcp-authz-server`
DOCS	**Official MCP SDKs for every language** · MCP	`https://modelcontextprotocol.io/docs/sdk`

MY NOTES FOR THIS WEEK

WEEK 04

LangGraph 1.x in Production

FULL-TIME · ~36 h
Suggested: 9–15 Nov

Goal. Master LangGraph's runtime (state, persistence, durable execution, human-in-the-loop, streaming, subgraphs and middleware) and move your old projects to LangChain/LangGraph 1.x.

Why it matters: You already know LangGraph basics. The production features are what separate a demo from a system that survives crashes and needs human approval.

HEADS-UP
LangChain 1.0 made `create_agent` the standard way to build agents. `AgentExecutor` and `create_react_agent` are deprecated, and all LangChain packages now need Python 3.10+.

DAY 1 **Migrate to 1.x**	• Read *What's new in LangChain v1* and both migration guides (LangChain and LangGraph). • Migrate the Travel Planner to `create_agent`. Log every change in `MIGRATION.md`. • Start LangChain Academy's *Introduction to LangGraph*. Run the notebooks, don't just watch.
DAY 2 **State & graph design**	• Graph API: typed state, reducers (write one custom reducer), conditional edges, `Command` (update + goto) and `Send` for map-reduce. • Build a document-processing graph: split → parallel analysis with `Send` → a reducer merges the results → final summary.
DAY 3 **Persistence & durable execution**	• Checkpointers: in-memory → SQLite → Postgres. Threads, `get_state`, state history, replay and fork (time travel), and durability modes. • Kill the process in the middle of a 10-step run and resume from the last checkpoint. Write down what re-runs and why side effects must be idempotent. • Add a long-term store for memory shared across threads.
DAY 4 **Human-in-the-loop & streaming**	• `interrupt()` + `Command(resume=...)`: approve, edit or reject a tool call. Then do the same with `HumanInTheLoopMiddleware` in `create_agent`. • Build an email-drafting agent where "send" (mocked) always needs your approval. • Stream tokens and progress events through a FastAPI SSE endpoint to a small React or Streamlit UI.
DAY 5 **Subgraphs & middleware**	• Put an agent inside a parent graph as a subgraph, and practise shared vs private state. • Use the built-in middleware (summarisation, model and tool call limits, retries). Write one custom middleware: a per-run cost budget that stops the agent gracefully.
DAY 6 **Octra v2 + wrap-up**	• Upgrade a copy of Octra: LangGraph 1.x, Postgres checkpointer, human approval on escalation, streaming and the cost-budget middleware. • Before/after architecture diagram, README, self-check. Push.

DELIVERABLE
Migrated Travel Planner, Octra v2 (persistence, human-in-the-loop, streaming, budgets) and `MIGRATION.md`.

SELF-CHECK: YOU ARE DONE WHEN YOU CAN…
- ☐ If the server restarts mid-run, what happens, and which code runs again when you resume?
- ☐ Checkpointer (one thread) vs store (across threads): when do you use each?
- ☐ `Command` vs conditional edges vs `Send`: when do you pick which?

Resources

COURSE	**Introduction to LangGraph (free)** · LangChain Academy	`https://academy.langchain.com`
DOCS	**What's new in LangChain v1** · LangChain docs	`https://docs.langchain.com/oss/python/releases/langchain-v1`
DOCS	**LangChain v1 migration guide** · LangChain docs	`https://docs.langchain.com/oss/python/migrate/langchain-v1`
DOCS	**LangGraph v1 migration guide** · LangChain docs	`https://docs.langchain.com/oss/python/migrate/langgraph-v1`
DOCS	**Graph API: state, reducers, Command, Send** · LangChain docs	`https://docs.langchain.com/oss/python/langgraph/graph-api`
DOCS	**Persistence** · LangChain docs	`https://docs.langchain.com/oss/python/langgraph/persistence`
DOCS	**Durable execution** · LangChain docs	`https://docs.langchain.com/oss/python/langgraph/durable-execution`
DOCS	**Interrupts (human-in-the-loop)** · LangChain docs	`https://docs.langchain.com/oss/python/langgraph/interrupts`
DOCS	**Middleware guide (built-in and custom)** · LangChain docs	`https://docs.langchain.com/oss/python/langchain/middleware`
DOCS	**Changelog (check it every Sunday)** · LangChain docs	`https://docs.langchain.com/oss/python/releases/changelog`

MY NOTES FOR THIS WEEK

WEEK 05

Multi-Agent Orchestration Patterns

FULL-TIME · ~36 h
Suggested: 16–22 Nov

Goal. Learn when several agents beat one (and when they don't) by building the main patterns and measuring them against each other.

Why it matters: Interviewers rarely ask "can you build multi-agent?". They ask "why multi-agent here?". This week gives you data-backed answers.

HEADS-UP
In Anthropic's data, multi-agent systems used about 15× the tokens of a normal chat. Always compare against a single-agent baseline before you keep extra agents.

DAY 1 **The debate: read first**	• Read Anthropic's *How we built our multi-agent research system*, Cognition's *Don't Build Multi-Agents* and its April 2026 follow-up *Multi-Agents: What's Actually Working*, and LangChain's *How and when to build multi-agent systems*. • Write a 1-page decision guide: single agent vs workflow vs multi-agent. Criteria: can the work run in parallel? does every agent need the same context? token budget? latency? who takes write actions?
DAY 2 **Router & handoffs**	• Router (you know it from Octra): structured output with a confidence score, a "clarify" path when confidence is low, and accuracy per route. • Handoffs: a support desk where a triage agent hands control to billing or technical agents, or to a human, using `Command`. Follow the official handoffs tutorial, then adapt it.
DAY 3 **Orchestrator-worker with parallel subagents**	• A lead agent splits a question into 3–5 subtasks. Each brief states the objective, output format, tools to use and boundaries (what not to do). • Workers run in parallel (`Send`) with their own context and step/token limits, return compressed findings, and the lead synthesises. • Measure on 10 questions against a single agent: quality (your rubric), tokens, cost, latency.
DAY 4 **Hierarchies, planning & failure modes**	• Build a small hierarchy (a supervisor over two teams) and a plan-and-execute agent that re-plans after failures, with a critic step. • Read *Why Do Multi-Agent LLM Systems Fail?* (MAST). Map its failure categories (specification problems, inter-agent misalignment, weak verification) to bugs you actually hit this week.
DAY 5 **Deep Agents**	• Build a research agent with `create_deep_agent` (planning tool, virtual filesystem, subagents), then read the source of each middleware to see how it works. • Read the open_deep_research code as a reference design for your capstone: supervisor subgraph, researcher subgraph and clarification step.
DAY 6 **Bake-off + wrap-up**	• Same 10 tasks, three architectures: single agent with tools, your orchestrator-worker, and a deep agent. Put quality, tokens, cost and latency in one table. • Write it up. It becomes your blog post in Week 11. Self-check. Push.

DELIVERABLE
Repo `week05-multi-agent`: router, handoffs, orchestrator-worker, hierarchy, deep agent, decision guide and bake-off results.

SELF-CHECK: YOU ARE DONE WHEN YOU CAN…

- ☐ Give two tasks where multi-agent wins and two where it loses, with reasons.
- ☐ List what every subagent brief must contain, and why.
- ☐ Name three MAST failure modes and how you would spot each one in traces.

Resources

READ	**How we built our multi-agent research system** · Anthropic	https://www.anthropic.com/engineering/multi-agent-research-system
READ	**Don't Build Multi-Agents (June 2025)** · Cognition	https://cognition.ai/blog/dont-build-multi-agents
READ	**Multi-Agents: What's Actually Working (April 2026 follow-up)** · Cognition	https://cognition.ai/blog/multi-agents-working
READ	**How and when to build multi-agent systems** · LangChain	https://www.langchain.com/blog/how-and-when-to-build-multi-agent-systems
DOCS	**Multi-agent patterns: subagents, handoffs, router, skills** · LangChain docs	https://docs.langchain.com/oss/python/langchain/multi-agent
DOCS	**Tutorial: handoffs for customer support** · LangChain docs	https://docs.langchain.com/oss/python/langchain/multi-agent/handoffs-customer-support
PAPER	**Why Do Multi-Agent LLM Systems Fail? (MAST taxonomy)** · Cemri et al., 2025	https://arxiv.org/abs/2503.13657
DOCS	**Deep Agents overview** · LangChain docs	https://docs.langchain.com/oss/python/deepagents/overview
CODE	**Deep Agents source code** · GitHub	https://github.com/langchain-ai/deepagents
CODE	**open_deep_research (archived Aug 2026; read it as a reference design)** · GitHub	https://github.com/langchain-ai/open_deep_research

MY NOTES FOR THIS WEEK

WEEK 06

Google ADK + A2A + Capstone Design

FULL-TIME · ~36 h
Suggested: 23–29 Nov

Goal. See the same ideas in a second framework, connect agents built in different frameworks with A2A, and design your capstone before writing its code.

Why it matters: Real companies mix frameworks and vendors. A2A lets agents that don't share code or memory work together, and a design doc stops you from building the wrong thing.

DAY 1 **ADK basics**	• ADK quickstart: `LlmAgent`, tools, sessions and state, callbacks. Then the workflow agents: `SequentialAgent`, `ParallelAgent` and `LoopAgent`. • Rebuild one Week 5 pattern in ADK. Write a short note on what is easier or harder than in LangGraph.
DAY 2 **ADK deeper**	• Multi-agent in ADK: sub-agents and transfers. Connect your Week 3 MCP server as a tool source. • Try ADK's built-in evaluation on 5 cases. You will go deeper in Week 8.
DAY 3 **The A2A protocol**	• Read the A2A v1.0 spec overview and *What's new in v1.0*: Agent Card, task lifecycle, messages, parts and artifacts, streaming, push notifications, auth and signed Agent Cards. • Follow ADK's *Exposing* quickstart: wrap an agent with `to_a2a()` and inspect the auto-generated Agent Card. Then follow the *Consuming* quickstart.
DAY 4 **Cross-framework connection**	• Build an ADK Localisation Agent (English → Gujarati/Hindi that keeps technical terms) and expose it over A2A. • Call it from a LangGraph orchestrator as a remote subagent. Handle timeouts, retries and auth between the two agents. • Optional (2 h): skim the OpenAI Agents SDK docs (agents, handoffs, guardrails, sessions, tracing) so you recognise its vocabulary.
DAY 5 **Capstone design doc (no code today)**	• Choose a domain you can judge yourself, because you will grade the answers in Week 8. See the Capstone Specification page for options. • Write a 2-page design doc: users, example tasks, architecture diagram, agents and their jobs, MCP tools and the A2A agent, state schema, human approval points, failure handling, eval plan with target metrics, threat model and cost budget per run.
DAY 6 **Capstone skeleton + wrap-up**	• Repo skeleton: FastAPI backend, LangGraph app, Postgres, Qdrant, Langfuse, a minimal UI, CI (lint + tests) and `.env.example`. • One `docker compose up` must start everything. Self-check. Push.

DELIVERABLE
ADK + A2A cross-framework demo, capstone design doc, and a capstone skeleton that starts with one command.

SELF-CHECK: YOU ARE DONE WHEN YOU CAN…

- ☐ Draw your capstone and mark which links are MCP and which are A2A, and explain why.
- ☐ Explain what an Agent Card contains and how an A2A task moves through its states.
- ☐ Name one thing ADK does better than LangGraph and one thing it does worse, from your own code.

Resources

DOCS	**Agent Development Kit (ADK) documentation** · Google `https://google.github.io/adk-docs/`
DOCS	**ADK with A2A: introduction** · Google `https://google.github.io/adk-docs/a2a/`

DOCS	**Quickstart: exposing an agent via A2A** · Google	https://google.github.io/adk-docs/a2a/quickstart-exposing
DOCS	**Quickstart: consuming a remote agent via A2A** · Google	https://google.github.io/adk-docs/a2a/quickstart-consuming
DOCS	**MCP tools in ADK** · Google	https://google.github.io/adk-docs/mcp/
DOCS	**A2A specification (v1.0)** · A2A Project	https://a2a-protocol.org/latest/specification/
READ	**What's new in A2A v1.0** · A2A Project	https://a2a-protocol.org/latest/whats-new-v1/
CODE	**ADK for Python: source and samples** · GitHub	https://github.com/google/adk-python
DOCS	**OpenAI Agents SDK (optional)** · OpenAI	https://openai.github.io/openai-agents-python/
TOOL	**Excalidraw for architecture diagrams** · Excalidraw	https://excalidraw.com

MY NOTES FOR THIS WEEK

WEEK 07

Capstone Build: Research Assistant

FULL-TIME · ~36 h
Suggested: 30 Nov–6 Dec

Goal. Build the capstone end to end, with tracing switched on from the first hour.

Why it matters: This one project pulls together everything from Weeks 1–6 and becomes the centre of your portfolio.

HEADS-UP
Get one question working end to end early, even with a single researcher, then widen. A thin working slice beats five half-built agents.

DAY 1 **Tracing first, then the lead agent**	• Self-host Langfuse with Docker Compose and connect LangGraph. Every LLM and tool call must show tokens, cost and latency. • Define the state schema. Build the lead agent: clarify the question → produce a structured research plan → pause for your approval (`interrupt`) → continue.
DAY 2 **Parallel researcher subagents**	• Fan out 3–5 researchers with `Send`. Each one gets a web-search tool (pick one search API with a free tier), a fetch/extract tool, its own context and step/token limits. • Each returns compressed findings with source URLs. A researcher that fails or times out must not fail the whole run.
DAY 3 **Documents & MCP**	• Upload path: PDFs → chunk → embed → Qdrant, plus a documents-researcher subagent. • Expose your research tools as an MCP server (reuse Week 3), and consume your Week 3 server where it fits.
DAY 4 **Writing, verification, localisation**	• A writer drafts the report with inline citations. A verifier checks each claim against its source and flags unsupported claims, with at most 2 revision rounds. • Call your ADK Localisation Agent over A2A for Gujarati/Hindi output.
DAY 5 **Reliability**	• Postgres checkpointer: prove crash-and-resume. Retries with backoff and a fallback model. • Per-run budgets (steps, tokens, money, time) enforced in middleware, with graceful partial results when a budget runs out.
DAY 6 **UI, first real test, wrap-up**	• Streaming UI that shows progress ("planning… 4 researchers running… verifying…"), a report view with clickable citations, and the cost and time of each run. • Run 10 real questions and read every trace. Write a backlog of failures for Week 8. Push.

DELIVERABLE
Capstone v1 running locally with one `docker compose up`: plan approval, parallel researchers, MCP tools, A2A localisation, verification, crash-resume and full tracing.

SELF-CHECK: YOU ARE DONE WHEN YOU CAN…
- ☐ Open any run in Langfuse and explain every step, its cost and its latency.
- ☐ Kill the backend mid-run: does it resume from the last checkpoint?
- ☐ What happens when one researcher fails?

Resources

DOCS **Self-hosting Langfuse (Docker Compose, Kubernetes)** · Langfuse
https://langfuse.com/self-hosting

DOCS **Tracing LangGraph with Langfuse** · Langfuse
https://langfuse.com/integrations/frameworks/langgraph

CODE	**Langfuse source and Docker Compose quick start** · GitHub	https://github.com/langfuse/langfuse
DOCS	**Graph API: Send for parallel fan-out** · LangChain docs	https://docs.langchain.com/oss/python/langgraph/graph-api
CODE	**open_deep_research: reference architecture (archived)** · GitHub	https://github.com/langchain-ai/open_deep_research
DOCS	**OpenTelemetry semantic conventions for GenAI (optional)** · OpenTelemetry	https://opentelemetry.io/docs/specs/semconv/gen-ai/
DOCS	**Tavily: one search-API option (check its current free tier)** · Tavily	https://docs.tavily.com

MY NOTES FOR THIS WEEK

WEEK 08

Evals & Observability

FULL-TIME · ~36 h
Suggested: 7–13 Dec

Goal. Turn "it seems to work" into numbers you can defend, and make bad changes fail CI.

Why it matters: This is your biggest gap today, and it is what turns a demo builder into a production engineer.

HEADS-UP
This is the most important week of the plan. If you are behind, cut scope anywhere else, but not here.

DAY 1 **Error analysis**	• Read Anthropic's *Demystifying evals for AI agents* and Hamel Husain's evals FAQ. • Review 50–100 capstone traces. Write a one-line note for each failure (open coding), group the notes into categories (axial coding) and count them. This ranked list decides what you measure.
DAY 2 **Golden datasets**	• Capstone: 50 questions with reference facts or sources and a pass/fail rubric. Octra: 100 labelled queries for routing and 30 for escalation. • Store them as Langfuse datasets, or as JSONL files versioned with DVC (which you already know).
DAY 3 **Evaluators**	• Code-based checks first: routing accuracy, valid citation URLs, required sections, step count, latency and cost. • LLM-as-judge for groundedness and completeness: a binary pass/fail with a written critique. Label 50 outputs yourself and measure how often the judge agrees with you (true-positive and true-negative rates). Fix the judge prompt until it does.
DAY 4 **Trajectory & component evals**	• Trajectory checks: did it search before writing? did the verifier run? any loops or wasted steps? • Consistency: run each task 3 times (pass^k). Component evals: router accuracy, retriever recall@k and verifier precision.
DAY 5 **CI & online evals**	• GitHub Actions: a fast eval subset (about 20 cases) on every pull request, with thresholds that fail the build, and the full suite nightly or on demand. • Online: LLM-judge scores on a sample of live traces, thumbs-up/down feedback stored in Langfuse, and a dashboard.
DAY 6 **Improve & report**	• Fix the top 3 failure categories (prompt, tool or architecture changes). Re-run the evals and record a before/after table for the capstone and for Octra. • Update your resume bullets with these real numbers. Push.

DELIVERABLE
Eval suite, CI gate, online eval dashboard, and a before/after report for both the capstone and Octra.

SELF-CHECK: YOU ARE DONE WHEN YOU CAN…
- ☐ Why binary pass/fail instead of 1–10 scores?
- ☐ How do you know your LLM judge can be trusted?
- ☐ What is the difference between pass@k and pass^k, and which one matters to users?

Resources

READ **Demystifying evals for AI agents (Jan 2026)** · Anthropic
https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents

READ **AI evals hub: start with "Your AI Product Needs Evals" and the "Field Guide"** · Hamel Husain
https://hamel.dev/notes/llm/evals/

READ	**LLM Evals FAQ** · Hamel Husain and Shreya Shankar	https://hamel.dev/blog/posts/evals-faq/
READ	**Evaluating the effectiveness of LLM-evaluators (LLM-as-judge)** · Eugene Yan	https://eugeneyan.com/writing/llm-evaluators/
PAPER	**τ-bench: tool-agent-user benchmark (introduces pass^k)** · Yao et al., 2024	https://arxiv.org/abs/2406.12045
DOCS	**Langfuse docs: datasets, experiments, LLM-as-a-judge** · Langfuse	https://langfuse.com/docs
COURSE	**Introduction to LangSmith (an alternative eval stack, free)** · LangChain Academy	https://academy.langchain.com
COURSE	**Agentic AI, Module 4: evals and error analysis** · Andrew Ng, DeepLearning.AI	https://www.deeplearning.ai/courses/agentic-ai/

MY NOTES FOR THIS WEEK

WEEK 09

Security & Guardrails

PART-TIME · ~10–12 h
Suggested: 14–20 Dec

Goal. Threat-model the capstone and harden it against prompt injection, tool misuse and data leaks.

Why it matters: An agent that reads untrusted content and can act is an attack surface, and interviewers know it.

HEADS-UP
New job: aim for about 1 hour on weekday evenings and 3–4 hours on weekend days.

SESSION 1 **2 h · Threat landscape**	• Read the OWASP Top 10 for Agentic Applications and Simon Willison's *lethal trifecta*. • Make a threat-model table: risk → where it appears → current defence → gap.
SESSION 2 **2 h · Attack yourself**	• Warm up with Gandalf, then attack your own capstone with indirect injection: a web page and a PDF that hide instructions. Record which attacks work, and why.
SESSION 3 **3 h · Defend by design**	• Read the *Design Patterns for Securing LLM Agents* paper (or Simon's summary). • Apply: least-privilege tools; treat fetched content as data, never as instructions; an allow-list for outbound fetches; human approval for anything irreversible; output validation; PII redaction.
SESSION 4 **2 h · Runtime limits & secrets**	• Budgets per run and per user, rate limits, secrets kept out of prompts and logs, audience checks on MCP tokens, and pinned dependency versions.
SESSION 5 **2 h · Make it permanent**	• Add 15–20 attack cases to your Week 8 eval suite so CI fails if an attack succeeds. Write `SECURITY.md`.

DELIVERABLE
Threat model, `SECURITY.md`, an attack test suite running in CI, and a hardened capstone.

SELF-CHECK: YOU ARE DONE WHEN YOU CAN…
- ☐ Explain the lethal trifecta and show where your capstone breaks one of its legs.
- ☐ Explain why "tell the model to ignore injected instructions" does not work.
- ☐ Say which actions in your system need human approval, and why exactly those.

Resources

READ	**OWASP Top 10 for Agentic Applications (2026)** · OWASP GenAI Security Project `https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications/`
READ	**OWASP Top 10 for LLM Applications** · OWASP GenAI Security Project `https://genai.owasp.org/llm-top-10/`
READ	**The lethal trifecta for AI agents** · Simon Willison `https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/`
READ	**Design patterns for securing LLM agents (summary)** · Simon Willison `https://simonwillison.net/2025/Jun/13/prompt-injection-design-patterns/`
PAPER	**Design Patterns for Securing LLM Agents against Prompt Injections** · Beurer-Kellner et al., 2025 `https://arxiv.org/abs/2506.08837`
TOOL	**Gandalf: a prompt-injection practice game** · Lakera `https://gandalf.lakera.ai`

WEEK 10

Cost, Reliability & Deployment

PART-TIME · ~10–12 h
Suggested: 21–27 Dec

Goal. Run the capstone like a real product: cheaper, sturdier, deployed and monitored.

Why it matters: This week plays to your existing strengths (Docker, Kubernetes, CI/CD, Prometheus/Grafana), now applied to agents.

SESSION 1 **2 h · Cost & latency**	• Measure cost per run by component. Send easy steps (routing, checks, verification) to a small model and keep the strong model for synthesis. • Turn on prompt caching for long, stable prompts, cap output lengths, then measure again.
SESSION 2 **2 h · Reliability**	• Provider fallback (your own small router, or a gateway such as LiteLLM with pinned versions), timeouts, retries with jitter, idempotent tools, a circuit breaker and graceful degradation.
SESSION 3 **3 h · Deploy**	• Containers for every service. Deploy to your existing Kubernetes/EKS setup (Langfuse has a Helm chart) or to a cheaper single VM (for example an OCI Always Free ARM VM with Docker Compose). Add HTTPS, secrets and Postgres backups.
SESSION 4 **2 h · Monitoring**	• Prometheus/Grafana for infrastructure and Langfuse for agent traces. Alerts on error rate, cost per day and drops in eval scores.
SESSION 5 **2 h · Read & reflect**	• Read Anthropic's *Effective harnesses for long-running agents* and *12-Factor Agents*. Write `LESSONS.md`: what you would change in your design now.

DELIVERABLE
A live deployment URL, dashboards and alerts, a before/after cost report, and `LESSONS.md`.

SELF-CHECK: YOU ARE DONE WHEN YOU CAN…
- ☐ What does one report cost now compared with Week 7, and which change saved the most?
- ☐ What happens when your main model provider is down?
- ☐ How would you roll back a bad prompt change in production?

Resources

READ	**Effective harnesses for long-running agents** · Anthropic	https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents
READ	**Harness design for long-running application development (Mar 2026)** · Anthropic	https://www.anthropic.com/engineering/harness-design-long-running-apps
READ	**12-Factor Agents: principles for production agents** · HumanLayer	https://github.com/humanlayer/12-factor-agents
DOCS	**Prompt caching** · Claude docs	https://platform.claude.com/docs/en/build-with-claude/prompt-caching
DOCS	**Self-hosting Langfuse (Docker, Kubernetes/Helm)** · Langfuse	https://langfuse.com/self-hosting
DOCS	**LiteLLM: model gateway with fallbacks** · LiteLLM	https://docs.litellm.ai
DOCS	**Oracle Cloud Free Tier** · Oracle	https://www.oracle.com/cloud/free/

WEEK 11

Portfolio & Visibility

PART-TIME · ~10–12 h
Suggested: 28 Dec–3 Jan

Goal. Make your work easy to find, quick to understand and credible.

Why it matters: Hiring in 2026 is skills-first. Proof of work with measured results beats a long list of keywords.

SESSION 1 **3 h · Capstone README**	• Problem, a 30-second demo GIF, architecture diagram, how it works, eval results table, security measures, cost per report, how to run it locally and known limitations.
SESSION 2 **2 h · Demo video (3–5 min)**	• Show one live run, its trace in Langfuse, the eval dashboard, and a prompt-injection attack being blocked.
SESSION 3 **3 h · Blog post**	• "What I learned building a multi-agent research assistant: evals, costs and failure modes". Use your Week 5 bake-off and Week 8 numbers. Publish it (Medium, Hashnode or dev.to) and share it on LinkedIn.
SESSION 4 **2 h · Resume & GitHub**	• Lead with your agent projects. Rewrite each bullet as action + method + measured result. Trim unrelated items, pin your best 4 repos and write a profile README.
SESSION 5 **1–2 h · Clean up**	• Tidy the Week 1–6 repos: READMEs, tests, and no secrets anywhere in the git history.

DELIVERABLE
A polished GitHub profile, demo video, published blog post and an updated resume.

SELF-CHECK: YOU ARE DONE WHEN YOU CAN…
☐ Can a stranger understand your capstone in 60 seconds from the README alone?
☐ Does every agent-related resume bullet include a measured number?

Resources

TOOL	**Excalidraw for clean architecture diagrams** · Excalidraw `https://excalidraw.com`
READ	**Your own Week 5 bake-off and Week 8 eval reports (your best source material)**

MY NOTES FOR THIS WEEK

WEEK 12

Review, System Design & What's Next

PART-TIME · ~10–12 h
Suggested: 4–10 Jan

Goal. Lock in what you learned, practise explaining designs, and choose your next focus.

Why it matters: Being able to explain trade-offs clearly is what makes people trust your designs at work.

SESSION 1 **2 h · Closed-book review**	• Answer every weekly self-check in writing without notes, then re-study the weak spots.
SESSION 2 **3 h · System design practice**	• 45 minutes each, on paper: (1) a support agent for an e-commerce company with refunds and escalation; (2) a coding agent for a large repository; (3) an invoice-processing agent at 100,000 documents a month. • For each one cover requirements, architecture, tools/MCP, memory, human approval, evals, security and cost.
SESSION 3 **2 h · Bring it to work**	• Find one real problem at your company where an agent or a simple workflow would help. Write a 1-page proposal: problem, approach, eval plan, risks and cost.
SESSION 4 **2 h · Choose your next focus**	• Pick one for the next 3 months: voice agents, computer-use and browser agents, coding agents and harness engineering, fine-tuning small models for tool calling, or managed agent platforms on the big clouds.
SESSION 5 **1 h · Next quarter's plan**	• Write a one-page plan for the next 12 weeks in the same format as this one.

DELIVERABLE
Three system-design write-ups, one work proposal and your next-quarter plan.

SELF-CHECK: YOU ARE DONE WHEN YOU CAN…
- ☐ Explain one of your designs in 5 minutes, including the trade-offs you rejected.
- ☐ Name the first thing you would measure in any new agent project, and why.

Resources

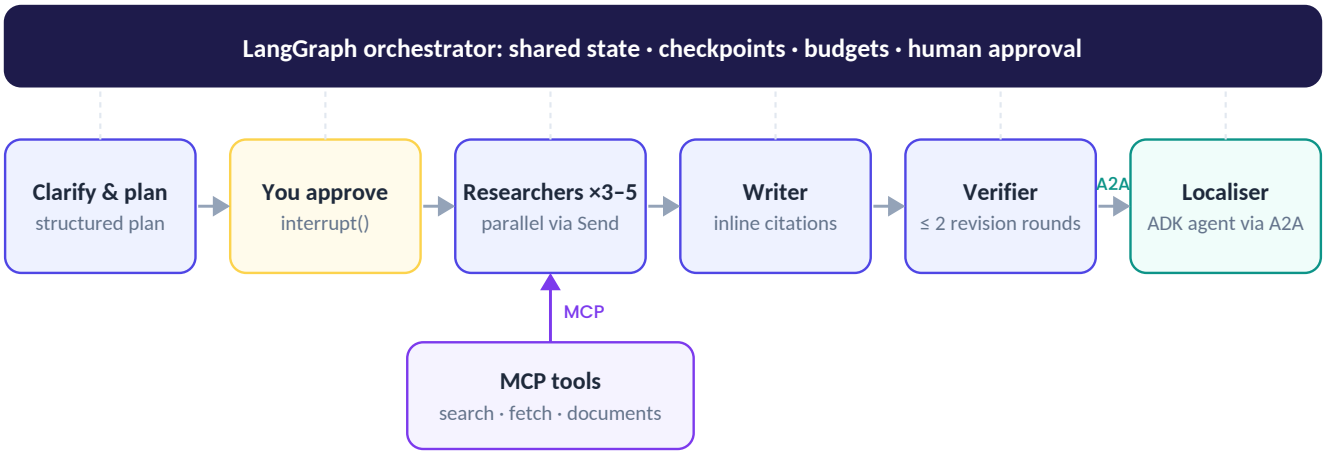
BOOK	**AI Engineering (see the agents and evaluation chapters)** · Chip Huyen	https://github.com/chiphuyen/aie-book
READ	**Anthropic Engineering blog (follow for new posts)** · Anthropic	https://www.anthropic.com/engineering
DOCS	**Claude Agent SDK overview (coding agents and harnesses)** · Claude docs	https://platform.claude.com/docs/en/agent-sdk/overview
DOCS	**LiveKit Agents (voice agents)** · LiveKit	https://docs.livekit.io/agents/
CODE	**Browser Use (browser agents)** · GitHub	https://github.com/browser-use/browser-use
READ	**Simon Willison's weblog (agents, security, new tools)** · Simon Willison	https://simonwillison.net

WEEKS 6–11

Capstone specification

One project ties the second half of the plan together: a **multi-agent research assistant**. Pick a domain you can judge yourself, because you will grade its answers in Week 8.

- **Indian SME market research:** market size, competitors and regulations for a business idea, with sources.
- **Tech due-diligence:** compare libraries or APIs for an engineering decision and produce a decision memo.
- **Government scheme navigator:** explain schemes for a given profile in Gujarati or Hindi, citing official sources.



Shared infrastructure: FastAPI + UI · Postgres (checkpoints) · Qdrant (vectors) · Langfuse (traces, evals) · guardrails

MUST-HAVE FEATURES

- ☐ Clarify → plan → you approve the plan (`interrupt`).
- ☐ Lead agent + 3–5 parallel researchers, each with its own context and step/token limits.
- ☐ Tools over MCP: web search, fetch/extract, document search (Qdrant).
- ☐ One remote specialist over A2A (ADK Localisation Agent).
- ☐ Writer + verifier: every claim cited; unsupported claims flagged.
- ☐ Durable execution: Postgres checkpointer, resume after a crash.
- ☐ Budgets per run: steps, tokens, money and time; graceful partial results.
- ☐ Every run traced in Langfuse with cost and latency.
- ☐ Evals: 50+ case golden set, validated LLM judge, CI gate, online evals.
- ☐ Security: threat model, attack suite in CI, least privilege, outbound allow-list.
- ☐ Deployed with Docker on Kubernetes or a VM, with dashboards and alerts.
- ☐ README with demo GIF, diagram, eval table, cost per report and limitations.

Suggested stack: Python 3.12 · LangGraph/LangChain 1.x · Google ADK (A2A specialist) · MCP Python SDK v2 · FastAPI · Postgres · Qdrant · Langfuse · React or Streamlit · Docker · GitHub Actions · Kubernetes (EKS) or a single VM.

Example targets for your design doc (set your own): task success of 80% or more on the golden set, citation accuracy of 90% or more, zero successful attacks in the security suite, p95 time per report under 5 minutes, and a fixed cost ceiling per report.

WEEK 12 AND BEYOND

Final checklist

Tick these off as you go. If every box is ticked by mid-January, your portfolio will show production agent skills, not just demos.

- ☐ **W1** Agent loop and the 5 workflow patterns, with cost and latency numbers
- ☐ **W2** Context and memory experiments, each with a before/after number
- ☐ **W3** Remote, OAuth-protected MCP server on the 2026-07-28 spec, plus your own client
- ☐ **W4** Travel Planner and Octra migrated to LangGraph/LangChain 1.x
- ☐ **W5** Multi-agent bake-off report (single vs orchestrator-worker vs deep agent)
- ☐ **W6** ADK agent connected over A2A, and the capstone design doc
- ☐ **W7** Capstone v1 running locally with full tracing
- ☐ **W8** Eval suite with a CI gate and before/after numbers on your resume
- ☐ **W9** Threat model and attack suite running in CI
- ☐ **W10** Live deployment, dashboards and a cost report
- ☐ **W11** README, demo video, published blog post, updated resume
- ☐ **W12** Three system-design write-ups, a work proposal and your next-quarter plan

If you fall behind

- Never skip Week 8. A smaller capstone with real evals beats a bigger one without them.
- If work gets busy, merge Weeks 9 and 10. Keep the attack suite and the deployment.
- Drop optional items first: MCP Apps, the OpenAI Agents SDK skim, extra UI polish.

Staying current after Week 12

- Every Sunday, spend 15 minutes on the LangChain changelog, the MCP blog and the Anthropic Engineering blog. Agent tooling changes every few months, and this plan will age too.
- Before trusting any tutorial, check its date and the library version it uses.
- Keep one small side project alive so you always have a place to try new ideas.

> ONE LAST TIP
> When you are unsure what to do next, look at your traces. They will tell you more than any new framework or course.